

# ApunteCódigosEjemplo01

## Guía de Explicación de Código y Métodos para Node.js — Programación III

**Propósito de este documento:** Este apunte documenta y explica brevemente los scripts de JavaScript (Node.js) incluidos en el directorio CódigosEjemplo01 del repositorio del curso. Su objetivo es servir como material de referencia para las clases prácticas, analizando qué hace cada script, cuáles son sus salidas esperadas por consola y el funcionamiento exacto de las funciones y métodos utilizados en cada uno.

### 1. typeof.js: Inferencia de Tipos, Reasignación Dinámica y Coerción Implícita

Demuestra las características fundamentales del sistema de tipos de JavaScript en Node.js, contrastándolas con lenguajes de tipado estático como C y C#. Muestra cómo el tipo de dato pertenece al valor y no a la variable, permitiendo la reasignación dinámica, y ejemplifica el fenómeno de la coerción implícita de tipos durante operaciones aritméticas y de concatenación.

#### Código de Ejemplo: typeof.js

```
// 1. Declaración con inferencia de tipo inicial
let x = 20000;
console.log("Valor:", x, "| Tipo:", typeof x);
// Output: Valor: 20000 | Tipo: number

// 2. Reasignación con otro tipo de dato (Imposible en C ó C#)
x = "Ahora soy una cadena";
console.log("Valor:", x, "| Tipo:", typeof x);
// Output: Valor: Ahora soy una cadena | Tipo: string

// 3. Operaciones raras por coerción implícita de tipos
let resultado1 = "5" + 2;
let resultado2 = "5" - 2;
console.log("");

console.log("Resultado '5' + 2:", resultado1, "| Tipo:", typeof resultado1);
console.log("Resultado '5' - 2:", resultado2, "| Tipo:", typeof resultado2);
```

#### ¿Qué hace el código?

- Inicializa la variable x con un valor numérico (20000) e inspecciona su tipo mediante typeof.
- Reasigna x con una cadena de texto ('Ahora soy una cadena'), demostrando que el tipado dinámico permite cambiar el tipo alojado en tiempo de ejecución.
- Realiza la operación '5' + 2, donde el operador + actúa como concatenador, convirtiendo automáticamente el número 2 a '2' y retornando la cadena '52'.
- Realiza la operación '5' - 2, donde el operador - no existe para cadenas de texto, forzando la coerción implícita de la cadena '5' al número 5 y devolviendo el resultado numérico 3.

#### Funciones y Métodos Utilizados:

| Elemento | Tipo              | Explicación y Uso en Node.js                                                                            |
|----------|-------------------|---------------------------------------------------------------------------------------------------------|
| let      | Palabra Reservada | Declara una variable reasignable con ámbito de bloque (block scope).                                    |
| typeof   | Operador Unario   | Devuelve una cadena que indica el tipo de dato del valor evaluado en tiempo de ejecución (ej: 'number', |

|               |               |                                                                                    |
|---------------|---------------|------------------------------------------------------------------------------------|
|               |               | 'string').                                                                         |
| console.log() | Método global | Imprime mensajes y datos formateados en la salida estándar de la consola (stdout). |

## 2. problemaVar.js:

### Ámbito de Alcance (Scope) y Comportamiento de var vs. let y const

Analiza el alcance de las variables en JavaScript. Demuestra cómo las variables declaradas con var ignoran las estructuras de control como if y se 'fugan' al scope superior, mientras que let y const respetan el alcance de bloque delimitado por llaves { }.

#### Código de Ejemplo: problemaVar.js

```
// Código JS usando var
if (true) {
  var mensaje = "Hola desde el bloque";
}
// En C#, 'mensaje' no existiría fuera del if.
// En JS con var, la variable se "fuga" al scope exterior:
console.log(mensaje); // Imprime: Hola desde el bloque

if (true) {
  let bloqueSeguro = "Solo vivo dentro del IF";
  const constante = 100;
}

// Intentar acceder afuera lanza un error en tiempo de ejecución:
// console.log(bloqueSeguro); // ReferenceError: bloqueSeguro is not defined
```

#### ¿Qué hace el código?

- Declara una variable mensaje usando var dentro de una estructura if (true).
- Accede a la variable mensaje fuera de dicho bloque, imprimiendo exitosamente 'Hola desde el bloque' debido a que var posee scope de función o global.
- Declara variables con let y const dentro de un segundo bloque if.
- Documenta que el intento de acceder a bloqueSeguro fuera del bloque desencadenará un ReferenceError, garantizando encapsulamiento.

#### Funciones y Métodos Utilizados:

| Elemento | Tipo                         | Explicación y Uso en Node.js                                                                                                                         |
|----------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| var      | Palabra Reservada (Obsoleta) | Declara variables con alcance de función o global. Sufre elevación (hoisting) e ignora bloques condicionales y bucles. Se debe evitar en JS moderno. |
| let      | Palabra Reservada            | Declara variables reasignables con alcance limitado estrictamente al bloque { } donde residen.                                                       |
| const    | Palabra Reservada            | Declara constantes con alcance de bloque. Impide la reasignación de la referencia en memoria.                                                        |

### 3. usoMap.js: Transformación Inmutable de Colecciones con .map() (Comparación Imperativa vs. Funcional)

Presenta la transformación de arreglos mediante la aplicación de impuestos (IVA del 21%). Compara el enfoque tradicional/imperativo utilizado en C/C# (mediante bucle for y mutación manual) con el paradigma funcional declarativo mediante el método .map().

#### Código de Ejemplo: usoMap.js

```
// =====  
// EJEMPLO 1: Calcular IVA (21%)  
// =====  
  
const precios = [100, 250, 500];  
  
// --- Enfoque Imperativo (C / C# tradicional) ---  
const preciosConIVAImperativo = [];  
for (let i = 0; i < precios.length; i++) {  
    preciosConIVAImperativo.push(precios[i] * 1.21);  
}  
  
// --- Enfoque Funcional (JavaScript) ---  
const preciosConIVAFuncional = precios.map(precio => precio * 1.21);  
  
// --- Salidas por consola ---  
console.log('--- ENTRADA ---');  
console.log('Precios originales:', precios);  
  
console.log('\n--- SALIDAS ---');  
console.log('Imperativo (for) :', preciosConIVAImperativo);  
console.log('Funcional (map)  :', preciosConIVAFuncional);  
  
// Comprobación de Inmutabilidad  
console.log('\n¿El arreglo original se modificó?:', precios);
```

#### ¿Qué hace el código?

- Crea un arreglo base de valores numéricos llamados precios.
- Calcula los precios con el 21% de IVA iterando imperativamente con un bucle for y agregando elementos a una nueva lista mediante .push().
- Calcula los mismos precios de forma funcional mediante precios.map(precio => precio \* 1.21) en una sola línea de código limpia.
- Imprime ambas listas finales y verifica que el arreglo original precios permanece inalterado ([100, 250, 500]), garantizando la inmutabilidad.

#### Funciones y Métodos Utilizados:

| Elemento               | Tipo              | Explicación y Uso en Node.js                                                                                                                               |
|------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Array.prototype.map()  | Método de Arreglo | Crea un nuevo arreglo con los resultados de la llamada a la función provista aplicada a cada elemento del arreglo original. No muta el arreglo de entrada. |
| Array.prototype.push() | Método Mutador    | Añade uno o más elementos al final de un arreglo y devuelve la nueva longitud del mismo.                                                                   |

```
precio => precio * 1.21
```

Arrow Function

Sintaxis concisa para expresar funciones anónimas. En este caso posee un retorno implícito de la multiplicación.

## 4. errorComun.js: Patrones Correctos e Incorrectos. Diferencias entre .map() y .forEach()

Explicita un error conceptual clásico en estudiantes que inician con JavaScript: utilizar .map() como si fuera un bucle de iteración sin devolver valores, resultando en arreglos llenos de undefined. Establece la regla clara de cuándo usar .map() frente a .forEach().

### Código de Ejemplo: errorComun.js

```
// =====
// EJEMPLO 4: Error común vs Uso Correcto
// =====

const numeros = [1, 2, 3, 4];

// ❌ ERROR COMÚN: Usar .map() sin `return` tratando de mutar algo externo
const malUsoMap = numeros.map(n => {
  n * 2; // Al no haber 'return', mapea cada valor a 'undefined'
});

console.log('❌ Resultado de map sin return:', malUsoMap);

// ✅ USO CORRECTO DE .map(): Retornar la expresión directa
const mapeoCorrecto = numeros.map(n => n * 2);
console.log('✅ Resultado de map correcto:', mapeoCorrecto);

// ✅ USO CORRECTO DE .forEach(): Para iterar y ejecutar acciones (Side-effects)
console.log('\n✅ Uso adecuado de forEach (imprimir paso a paso):');
numeros.forEach((n, i) => {
  console.log(`Índice ${i}: El doble de ${n} es ${n * 2}`);
});
```

### ¿Qué hace el código?

- Muestra el mal uso de .map(): al abrir llaves { n \* 2; } sin escribir la instrucción return, JavaScript devuelve implícitamente undefined para cada elemento. Salida: [undefined, undefined, undefined, undefined].
- Muestra el uso correcto de .map(): con retorno implícito en una sola línea (n => n \* 2), devolviendo la nueva colección proyectada: [2, 4, 6, 8].
- Muestra el uso correcto de .forEach(): iterar la lista para realizar efectos secundarios (en este caso, imprimir en pantalla) accediendo opcionalmente tanto al valor (n) como al índice (i).

### Funciones y Métodos Utilizados:

| Elemento                  | Tipo                | Explicación y Uso en Node.js                                                                                                                                                                                |
|---------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Array.prototype.forEach() | Método de Iteración | Ejecuta la función indicada una vez por cada elemento del arreglo. Siempre devuelve undefined. Se debe usar exclusivamente para efectos secundarios (imprimir, escribir archivos, mutar sistemas externos). |

(n, i) => ...

Callback Signature

La función callback de los métodos de arreglo en JS puede recibir hasta 3 argumentos opcionales: el elemento actual (n), el índice del elemento (i) y el arreglo completo.

## 5. usoSpread.js: Proyección y Enriquecimiento de Objetos mediante .map() y Operador Spread (...)

Ilustra dos de las operaciones más frecuentes en desarrollo Backend con Node.js y Frontend con React/React Native: la proyección (extracción selectiva de propiedades) y el enriquecimiento (agregado de nuevas propiedades calculadas sin mutar la estructura original).

### Código de Ejemplo: usoSpread.js

```
// =====  
// EJEMPLO 2: Transformación de Objetos  
// =====  
  
const estudiantes = [  
  { id: 1, nombre: 'Ana', nota: 8 },  
  { id: 2, nombre: 'Luis', nota: 5 },  
  { id: 3, nombre: 'Carlos', nota: 9 }  
];  
  
console.log('=== DATOS ORIGINALES ===');  
console.table(estudiantes);  
  
// A) Proyección: Extraer solo los nombres (Arreglo de strings)  
const nombres = estudiantes.map(estudiante => estudiante.nombre);  
  
console.log('\n--- 1. Solo nombres (Proyección) ---');  
console.log(nombres);  
  
// B) Enriquecimiento: Agregar propiedad 'aprobado' sin mutar los objetos originales  
const estudiantesConEstado = estudiantes.map(estudiante => ({  
  ...estudiante,           // Copia las propiedades actuales  
  aprobado: estudiante.nota >= 6 // Agrega la nueva propiedad  
}));  
  
console.log('\n--- 2. Estudiantes con estado de aprobación ---');  
console.table(estudiantesConEstado);  
  
// Verificación de inmutabilidad en el primer estudiante original  
console.log('\nVerificación: ¿El estudiante original sigue intacto?');  
console.log(estudiantes[0]);
```

### ¿Qué hace el código?

- Muestra un arreglo de objetos complejos estudiantes utilizando console.table() para un formato tabular visual en consola.
- Proyección: Mapea cada objeto para retornar únicamente el valor de la propiedad nombre, obteniendo un arreglo simple de cadenas: ['Ana', 'Luis', 'Carlos'].
- Enriquecimiento Inmutable: Genera un nuevo arreglo de objetos clonando las propiedades existentes con el operador spread (...estudiante) e incorporando la propiedad boolean aprobada: estudiante.nota >= 6.

- Comprueba que el objeto original `estudiantes[0]` no fue afectado ni posee la propiedad `aprobado`.

### Funciones y Métodos Utilizados:

| Elemento                           | Tipo                 | Explicación y Uso en Node.js                                                                                                                                        |
|------------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>console.table()</code>       | Método global        | Muestra datos estructurados (arreglos u objetos) en forma de tabla legible en la consola de Node.js.                                                                |
| <code>...</code> (Spread Operator) | Operador de Sintaxis | Desestructura y expande las propiedades iterables de un objeto o arreglo dentro de un nuevo contenedor, permitiendo realizar copias superficiales (shallow copies). |
| <code>({ ... })</code>             | Sintaxis de retorno  | Los paréntesis envuelven el cuerpo de la Arrow Function para permitir el retorno implícito de un literal de objeto sin confundirlo con el bloque de la función.     |

## 6. renderHTML.js: Generación Dinámica de Código / UI (HTML Templating con .map y .join)

Demuestra cómo transformar un conjunto de datos (strings) en estructuras de marcado como etiquetas HTML utilizando `.map()` e Interpolación de Cadenas (Template Literals), simulando la creación dinámica de componentes en servidores web o interfaces frontend.

### Código de Ejemplo: renderHTML.js

```
// =====
// EJEMPLO 3: Generación de Marcado / UI
// =====

const opcionesMenu = ['Inicio', 'Productos', 'Servicios', 'Contacto'];

// Transforma cada string en un string formateado como <li>
const itemsHTML = opcionesMenu.map(opcion => `<li>${opcion}</li>`);

// Unimos el arreglo generado en un solo string
const menuHTML = `<ul class="nav-menu">\n${itemsHTML.join('\n')}\n</ul>`;

console.log('--- ESTRUCTURA ENTRADA ---');
console.log(opcionesMenu);

console.log('\n--- SALIDA HTML GENERADA ---');
console.log(menuHTML);
```

### ¿Qué hace el código?

- Parte de un arreglo `opcionesMenu` con las secciones de un sitio web.
- Aplica `.map()` para envolver cada nombre de opción dentro de elementos HTML `<li>` utilizando comillas invertidas y la sintaxis `${opcion}`.
- Aplica `.join('\n')` sobre el arreglo resultante para concatenar los elementos en una única cadena separada por saltos de línea.

- Envuelve las líneas generadas dentro de un contenedor `<ul class="nav-menu">`, produciendo un bloque HTML listo para renderizarse o enviarse en una respuesta HTTP.

#### Funciones y Métodos Utilizados:

| Elemento                               | Tipo               | Explicación y Uso en Node.js                                                                                                                                   |
|----------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Template Literals ( <code>` `</code> ) | Sintaxis de String | Permite crear cadenas multilinea e interpolar expresiones o variables de forma concisa mediante <code>\${expresion}</code> .                                   |
| <code>Array.prototype.join()</code>    | Método de Arreglo  | Une todos los elementos de un arreglo en una cadena de texto única, insertando el separador especificado (en este caso, un salto de línea <code>'\n'</code> ). |

## 7. usoFilter.js: Filtrado de Colecciones con `.filter()`

Muestra cómo seleccionar y extraer elementos de un arreglo basándose en una o más condiciones lógicas que deben evaluarse como verdaderas (`true`).

#### Código de Ejemplo: usoFilter.js

```
// =====
// EJEMPLO: Uso de .filter()
// =====

const productos = [
  { id: 1, nombre: 'Teclado Mecánico', precio: 80, stock: true },
  { id: 2, nombre: 'Mouse Óptico', precio: 25, stock: false },
  { id: 3, nombre: 'Monitor 24"', precio: 200, stock: true },
  { id: 4, nombre: 'Auriculares', precio: 50, stock: true }
];

console.log('=== CATÁLOGO COMPLETO ===');
console.table(productos);

// Filtrar productos en stock y con precio mayor a $30
const disponiblesYValiosos = productos.filter(p => p.stock && p.precio > 30);

console.log('\n--- Productos en stock y > $30 (filter) ---');
console.table(disponiblesYValiosos);
```

#### ¿Qué hace el código?

- Define un arreglo de objetos representando un catálogo de productos informáticos.
- Aplica el método `.filter()` evaluando una condición compuesta: que el producto tenga stock disponible (`p.stock === true`) Y que su precio sea mayor a 30 (`p.precio > 30`).
- Devuelve un nuevo arreglo conteniendo únicamente los objetos que satisfacen ambas condiciones (Teclado Mecánico, Monitor 24" y Auriculares), excluyendo el Mouse Óptico.

#### Funciones y Métodos Utilizados:

| Elemento                              | Tipo              | Explicación y Uso en Node.js                                                                                 |
|---------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------|
| <code>Array.prototype.filter()</code> | Método de Arreglo | Crea un nuevo arreglo lleno con todos los elementos que pasan la prueba implementada por la función dada. Si |

|                 |                 |                                                                            |
|-----------------|-----------------|----------------------------------------------------------------------------|
|                 |                 | ningún elemento aprueba la condición, devuelve un arreglo vacío [].        |
| && (AND Lógico) | Operador Lógico | Evalúa dos expresiones y devuelve true únicamente si ambas son verdaderas. |

## 8. combina.js: Pipeline Funcional Completo (.filter + .map + .reduce)

Integra los tres métodos funcionales más importantes de JavaScript en una cadena o tubería (pipeline) de transformación de datos. Simula el procesamiento contable de transacciones bancarias o de comercio electrónico.

### Código de Ejemplo: combina.js

```
// =====
// EJEMPLO COMBINADO: Pipeline Funcional
// =====

const transacciones = [
  { id: 'T1', cliente: 'Juan', monto: 1500, estado: 'APROBADA' },
  { id: 'T2', cliente: 'María', monto: 3000, estado: 'RECHAZADA' },
  { id: 'T3', cliente: 'Pedro', monto: 800, estado: 'APROBADA' },
  { id: 'T4', cliente: 'Ana', monto: 2200, estado: 'PENDIENTE' },
  { id: 'T5', cliente: 'Sofia', monto: 1200, estado: 'APROBADA' }
];

console.log('=== TRANSACCIONES REGISTRADAS ===');
console.table(transacciones);

// --- PIPELINE DE TRANSFORMACIÓN FUNCIONAL ---
const ingresoNetoTotal = transacciones
  .filter(t => t.estado === 'APROBADA')
  .map(t => t.monto * 0.90)
  .reduce((acumulador, montoConDescuento) => acumulador + montoConDescuento, 0);

// --- Salidas paso a paso para inspección en clase ---
const aprobadas = transacciones.filter(t => t.estado === 'APROBADA');
const montosConDescuento = aprobadas.map(t => t.monto * 0.90);

console.log('\n--- DESGLOSE DEL PIPELINE ---');
console.log('1. Transacciones aprobadas (filter)   :', aprobadas.map(t => t.id));
console.log('2. Montos con 10% desc. (map)          :', montosConDescuento);
console.log('3. Total final acumulado (reduce)           :', `${ingresoNetoTotal}`);

console.log('\nVerificación: ¿El arreglo de transacciones original sufrió mutación?');
console.log(`Cantidad original intacta: ${transacciones.length} elementos.`);
```

### ¿Qué hace el código?

- Paso 1 (.filter): Selecciona del listado solo aquellas transacciones cuyo estado sea 'APROBADA' (T1, T3 y T5).
- Paso 2 (.map): Toma las transacciones aprobadas y aplica un descuento del 10% calculando el valor neto (monto \* 0.90). Transforma los objetos en un arreglo numérico: [1350, 720, 1080].
- Paso 3 (.reduce): Acumula y suma todos los montos procesados comenzando desde un valor inicial de 0, dando como resultado un único valor numérico: 3150.



- Imprime el desglose etapa por etapa para análisis pedagógico en clase y confirma que la colección original de transacciones sigue intacta.

**Funciones y Métodos Utilizados:**

| Elemento                 | Tipo              | Explicación y Uso en Node.js                                                                                                                                                                                                     |
|--------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Array.prototype.reduce() | Método Acumulador | Ejecuta una función 'reductora' sobre cada elemento del arreglo, devolviendo como resultado un único valor acumulado (un número, una cadena, un objeto o un nuevo arreglo). Sintaxis: .reduce((acum, val) => ..., valorInicial). |
| Method Chaining          | Patrón de Diseño  | Encadenamiento de métodos donde la salida de una función (.filter devuelve un arreglo) se conecta directamente como la entrada del siguiente método (.map).                                                                      |